Unit II

2. String Handling, Classes, Modules and Package

2.1 Strings, String operations and String Slicing
2.2 Defining Classes
2.3 Defining and calling functions passing arguments to functions
2.4 Python and OOP – Inheritance, polymorphism
2.5 Modules – datetime, math
2.6 Packages

2.1 Strings, String operations and String Slicing:

1. Strings:

A **string** in Python is a sequence of characters enclosed in single quotes (`' '`), double quotes (`" "`), or triple quotes (`''' '''` or `""" """`). Strings are used to store text data such as names, messages, and sentences.

Characteristics:

1. **Immutable:**
   Once created, the characters in a string cannot be changed.

2. **Indexed:**
   Each character has a position number (index), starting from 0 for the first character.

3. **Ordered Sequence:**
   Python preserves the order of characters as they appear.

4. **Supports Unicode:**
   Strings can store letters, numbers, symbols, even emojis.

5. **Multiline Strings:**
   Triple quotes allow storing multi-line text.

Example:

```
name = "Python"
```

Strings are widely used for input/output operations, file handling, and data processing.

2. String Operations:

Python provides various operations to manipulate and work with strings.

Common String Operations:

1. **Concatenation (`+`):**
   Joins two strings.
   Example: `"Hello" + "World"`

2. **Repetition (`*`):**
   Repeats a string.
   Example: `"Hi" * 3 → "HiHiHi"`

3. **Length (`len()`):**
   Returns number of characters.
   Example: `len("Python") → 6`

4. **Membership (`in`, `not in`):**
   Checks if a substring exists.
   Example: `"Py" in "Python"`

5. **String Methods:**
   `.upper()` → converts to uppercase
   `.lower()` → converts to lowercase
   `.replace()` → replaces part of a string
   `.find()` → returns the index of a substring
   `.split()` → converts string into list
   `.strip()` → removes spaces

String operations make text processing easy and powerful in Python.

3. String Slicing:

String slicing is the process of extracting a part (substring) from a string using the slice operator `[start:end:step]`.

Syntax:

string[start : end : step]

start: index from where slice begins
end: index where slice stops (excluded)
step: difference between each index (default 1)

Examples:

1. **Basic slicing:**
   s = "Python"
   s[0:3] → "Pyt"

2. **Omitting start or end:**
   s[:4] → "Pyth"
   s[2:] → "thon"

3. **Negative indexing:**
   s[-3:] → "hon"

4. **Using step:**
   s[::2] → "Pto"

String slicing is useful for extracting data, reversing strings, and manipulating text efficiently.

2.2 Defining Classes:

A **class** in Python is a user-defined blueprint used to create objects. It groups **data (attributes)** and **functions (methods)** into a single unit. Classes enable **Object-Oriented Programming (OOP)**, making programs modular, reusable, and easier to maintain.

Defining a Class:

A class is defined using the `class` keyword, followed by the class name and a block of statements.

Example:

```
class Student:
    def __init__(self, name, marks):
        self.name = name        # attribute
        self.marks = marks      # attribute

    def display(self):          # method
        print("Name:", self.name)
        print("Marks:", self.marks)
```

Key Points:

1. **`class` keyword:** Used to create a new class.
2. **Attributes:** Variables that store the state of an object.
3. **Methods:** Functions inside a class that define its behavior.
4. **`__init__` method:** Special constructor method used to initialize objects.
5. **Object Creation:**

```
s = Student("Ravi", 85)
s.display()
```

Conclusion:

Defining classes allows programmers to model real-world entities, implement reusability, and maintain structured, organized programs using OOP principles.

2.3 Defining and calling functions passing arguments to functions:

1. Defining Functions:

A **function** in Python is a reusable block of code that performs a specific task. Functions help reduce repetition, improve readability, and promote modular programming. Functions are defined using the `def` keyword.

Syntax:

```
def function_name(parameters):
    statement(s)
```

Key Points:

1. **`def` keyword** is used to define a function.
2. **Function name** should be meaningful and follow naming rules.
3. **Parameters** (optional) allow values to be passed.
4. A function may contain many statements.
5. The function can return a value using `return`.

Example:

```
def greet():
    print("Hello! Welcome to Python.")
```

Conclusion:

Defining functions helps in organizing programs into smaller, manageable parts, making the code reusable and easier to maintain.

2. Calling Functions:

Calling a function means executing the code defined inside the function. A function must be called after it is defined; otherwise, it will not run.

How to Call a Function:

Use the function name followed by parentheses.

Example:

```
def greet():
    print("Hello!")

greet()      # Function call
```

Key Points:

1. When a function is called, Python jumps to the function block and executes it.
2. After execution, the control returns to the calling statement.
3. Functions can be called multiple times in a program.
4. If the function requires parameters, they must be provided during the call.

Example with Argument:

```
def add(a, b):
    print(a + b)

add(5, 3)     # Calls function with values
```

Conclusion:

Calling functions allows code reuse and helps execute the same logic whenever required.

3. Passing Arguments to Functions:

Arguments are values supplied to a function when it is called. They help functions work on different data and produce different outputs.

Types of Arguments:

1. **Positional Arguments:**

   Passed in the same order as parameters.

```
def add(a, b):
    return a + b
add(10, 20)
```

2. **Keyword Arguments:**

   Provided using parameter names.

```
add(b=5, a=3)
```

3. **Default Arguments:**

   Parameters with default values.

```
def greet(name="Guest"):
    print("Hello", name)
```

4. **Variable-Length Arguments:**

   `*args` for multiple positional arguments

   `**kwargs` for multiple keyword arguments

Example:

```
def details(name, age=18):
    print(name, age)

details("Ravi", 20)
details("Asha")      # Uses default age
```

Conclusion:

Passing arguments allows functions to be flexible, reusable, and adaptable to different inputs.

2.4 Python and OOP – Inheritance, polymorphism:

1. Inheritance in Python:

Inheritance is an important Object-Oriented Programming (OOP) concept that allows a new class (child class) to acquire the properties and methods of an existing class (parent class). It promotes code reusability and supports hierarchical classification.

Key Points:

1. **Parent/Base Class:** The class whose properties are inherited.
2. **Child/Derived Class:** The class that receives attributes and methods.
3. **`super()` Function:** Used to access parent class features.
4. **Types of Inheritance in Python:**
   - Single inheritance
   - Multiple inheritance
   - Multilevel inheritance
   - Hierarchical inheritance
   - Hybrid inheritance

Example:

```
class Animal:
    def sound(self):
        print("Animal makes sound")

class Dog(Animal):    # Inheriting Animal
    def sound(self):
        print("Dog barks")

d = Dog()
d.sound()
```

Conclusion:

Inheritance helps reduce code duplication, supports extensibility, and allows building complex relationships among classes.

2. Polymorphism in Python:

Polymorphism means “many forms”. In OOP, it allows the same function name or operator to behave differently depending on the object or data type. Python supports polymorphism through method overriding, method overloading (limited), and operator overloading.

Forms of Polymorphism:

1. **Method Overriding:**
   A child class rewrites a method of the parent class.

2. **Duck Typing:**
If an object behaves like another, it can be used in its place.

3. **Operator Overloading:**
Same operator performs different actions (e.g., `+` works for numbers and strings).

Example — Method Overriding:

```
class Shape:
    def area(self):
        return 0

class Circle(Shape):
    def area(self):
        return 3.14 * 5 * 5
```

Example — Operator Overloading:

```
print(10 + 20)       # Addition
print("A" + "B")     # Concatenation
```

Conclusion:
Polymorphism increases flexibility and allows writing general-purpose code that works with different types of objects.

2.5 Modules – datetime, math:

1. datetime Module:

The `**datetime**` module in Python provides classes and functions to work with dates, times, and time intervals. It helps perform operations such as getting the current date/time, formatting dates, comparing times, and calculating differences.

Key Features:

1. **Date Handling:**
The `date` class allows creation and manipulation of dates (year, month, day).

2. **Time Handling:**
The `time` class stores time values like hours, minutes, seconds, and microseconds.

3. **DateTime Class:**
Combines both date and time into a single object using `datetime.datetime`.

4. **Timedelta:**
Represents the difference between two dates or times (e.g., age calculation, countdown).

5. **Formatting and Parsing:**
`strftime()` converts datetime into a string format; `strptime()` converts a string into datetime.

Example:

```
import datetime

now = datetime.datetime.now()
print(now.strftime("%d-%m-%Y %H:%M:%S"))
```

Conclusion:

The `datetime` module is essential for programs requiring scheduling, logging, timestamps, calendars, and time-based calculations.

2. math Module:

The **`math`** module provides mathematical functions and constants for performing advanced mathematical operations beyond basic arithmetic.

Key Features:

1. **Mathematical Functions:**
   Includes `sqrt()`, `pow()`, `ceil()`, `floor()`, `factorial()`, `log()`, etc.

2. **Trigonometric Functions:**
   `sin()`, `cos()`, `tan()`, `degrees()`, and `radians()` support trigonometric calculations.

3. **Constants:**
   Provides mathematical constants such as
   `math.pi` → value of π
   `math.e` → Euler's number

4. **Rounding and Absolute Operations:**
   Functions like `fabs()`, `trunc()`, `modf()` provide numeric manipulation.

5. **Precision and Performance:**
   Faster and more accurate than manually writing mathematical formulas.

Example:

```
import math

print(math.sqrt(25))
print(math.pi)
```

Conclusion:

The `math` module is widely used in scientific computing, engineering, data analysis, graphics, and applications requiring accurate mathematical computations.

2.6 Packages:

A **package** in Python is a way of organizing related modules into a directory structure. It helps in grouping similar functionalities together, making large programs easier to manage, maintain, and reuse. A package is simply a folder that contains multiple Python modules and an optional special file named **`__init__.py`**.

Key Features of Packages:

1. **Organized Structure:**
   Packages allow logical grouping of modules, similar to folders inside a computer.

2. **`__init__.py` File:**
   This file (even if empty) tells Python that the directory should be treated as a package. It can also contain initialization code.

3. **Nested Packages:**
   Packages can contain sub-packages (folders within folders), enabling hierarchical organization.

4. **Reusability:**
   Functions or modules from a package can be imported and reused across multiple programs.

5. **Importing Packages:**
   You can import a module from a package using:

```
import package_name.module_name
```

Example:

Folder structure:

```
mypackage/
    __init__.py
    calc.py
```

Using the package:

```
import mypackage.calc
```

Conclusion:

Packages make Python applications modular, scalable, and easier to develop. They are widely used in libraries such as NumPy, Pandas, Django, and many others.